

UART串行总线舵机Python SDK使用手册

创建串口总线舵机管理器

串口总线舵机的文件路径 `fashionstar-uart-servo-python/src/usersvo.py`，使用的时候可以将 `usersvo.py` 拷贝至你当前工程文件夹里面。

或者使用的时候，将 `usersvo.py` 所在的文件夹添加到系统路径里面，相对路径/绝对路径都可以。

```
# 添加usersvo.py的系统路径
import sys
sys.path.append("../src")
```

然后使用的过程中一般需要导入如下这两个依赖

```
# PySerial 负责串口总线通信
import serial
# UartServoManager 串口总线舵机管理器
from usersvo import UartServoManager
```

接下来要创建串口对象，指定相关的参数

```
# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                    baudrate=SERVO_BAUDRATE, \
                    parity=serial.PARITY_NONE, stopbits=1, \
                    bytesize=8, timeout=0)
```

创建舵机管理器，将串口对象传入到构造器 `UartServoManager` 里面。

```
# 初始化舵机管理器
userservo = UartServoManager(uart)
```

舵机通信检测

API-ping

调用舵机的 `ping()` 函数用于舵机的通信检测, 判断舵机是否在线。

函数原型

```
def ping(self, servo_id:int):
```

输入参数

- `servo_id` : 舵机ID

输出参数

- `is_online` : 舵机是否在线

例程源码

example/ping.py

```
'''
FashionStar Uart舵机
> Python SDK舵机通讯检测 Example <
-----
-
-----
'''

# 添加userservo.py的系统路径
import sys
sys.path.append("../src")
```

```

# 导入依赖
import time
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                      baudrate=SERVO_BAUDRATE, \
                      parity=serial.PARITY_NONE, stopbits=1, \
                      bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

# 舵机通讯检测
is_online = uservo.ping(SERVO_ID)
print("舵机ID={} 是否在线: {}".format(SERVO_ID, is_online))

```

舵机阻尼模式

API-set_damping

设置舵机为阻尼模式。

函数原型

```
def set_damping(self, servo_id, power=0):
```

输入参数

- `servo_id`: 舵机ID
- `power`: 舵机功率, 单位mW

输出参数

- 无

例程源码

```
'''
FashionStar Uart舵机
> Python SDK 舵机阻尼模式 <
-----
-
-----
'''

# 添加uservo.py的系统路径
import sys
sys.path.append("../../src")
# 导入依赖
import time
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                    baudrate=SERVO_BAUDRATE, \
                    parity=serial.PARITY_NONE, stopbits=1, \
                    bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

power = 500 # 阻尼模式下的功率, 单位mW
uservo.set_damping(SERVO_ID, power)
```

舵机角度查询

API-query_servo_angle

函数原型

```
def query_servo_angle(self, servo_id):
```

输入参数

- `servo_id`: 舵机ID

输出参数

- `angle`: 舵机角度(单圈/多圈)

注意事项

注意这里返回的角度是多圈模式的角度还是单圈模式的角度，取决于上次控制舵机的角度的指令是单圈模式还是/多圈模式，默认为单圈。

如果想人为的设定查询多圈/单圈，可以在查询之前设定 `uservo.servos[servo_id].is_mturn` 这个布尔值。

- `is_mturn=True`: 返回多圈角度
- `is_mturn=False`: 返回单圈角度

例程源码

设置舵机为阻尼模式，转动舵机 1s打印一下当前的角度。

`example/query_servo_angle.py`

```
'''
FashionStar Uart舵机
> Python SDK舵机角度查询 Example <
-----
-
-----
'''

# 添加uservo.py的系统路径
import sys
sys.path.append("../src")
# 导入依赖
import time
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
```

```

SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                      baudrate=SERVO_BAUDRATE, \
                      parity=serial.PARITY_NONE, stopbits=1, \
                      bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

# 设置舵机为阻尼模式
uservo.set_damping(SERVO_ID, 200)

# 舵机角度查询
while True:
    angle = uservo.query_servo_angle(SERVO_ID)
    print("当前舵机角度: {:.1f} °".format(angle), end='\r')
    time.sleep(1)

```

设置舵机角度

多圈角度模式仅限于磁编码系列舵机，其他系列不兼容该模式

API-set_servo_angle

设置舵机角度，这个API包含了6种舵机角度控制模式，通过传入不同的参数调用不同的指令。具体的使用方式，可以参考[例程源码](#)。

函数原型

```

def set_servo_angle(self, servo_id:int, angle:float,
                    is_mturn:bool=False, interval:float=None, velocity:float=None,
                    t_acc:int=20, t_dec:int=20, power:int=0, mean_dps:float=100.0):

```

输入参数

- `servo_id` : 舵机的ID号
- `angle` : 目标角度

- `is_mturn`: 是否是多圈模式
- `interval`: 中间间隔 单位ms
- `velocity`: 舵机的目标转速, 单位dps
- `t_acc`: 加速时间, 在指定目标转速时有效. 单位ms
- `t_dec`: 减速时间, 在指定减速时间时有效. 单位ms
- `power`: 功率限制, 单位mW
- `mean_dps`: 平均转速, 单位dps, 用于估计interval

输出参数

- 无

API-wait

等待所有的舵机到达目标角度。

函数原型

```
def wait(self, timeout=None):
```

输入参数

- `timeout`: 阻塞式等待的超时判断阈值, 单位ms

输出参数

- 无

例程源码

```
'''
FashionStar Uart舵机
> 设置舵机角度 <
-----
-
-----
'''
```

```

# 添加uservo.py的系统路径
import sys
sys.path.append(".././src")
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM6' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                    baudrate=SERVO_BAUDRATE, \
                        parity=serial.PARITY_NONE, stopbits=1, \
                        bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart, is_debug=True)

print("[单圈模式]设置舵机角度为90.0°")
uservo.set_servo_angle(SERVO_ID, 90.0, interval=0) # 设置舵机角度
# 极速模式
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))

print("[单圈模式]设置舵机角度为-80.0°，周期1000ms")
uservo.set_servo_angle(SERVO_ID, -80.0, interval=1000) # 设置舵机
# 角度(指定周期 单位ms)
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))

print("[单圈模式]设置舵机角度为70.0°，设置转速为200 °/s，加速时间
100ms，减速时间100ms")
uservo.set_servo_angle(SERVO_ID, 70.0, velocity=200.0,
                    t_acc=100, t_dec=100) # 设置舵机角度(指定转速 单位°/s)
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))

print("[单圈模式]设置舵机角度为-90.0°，添加功率限制")
uservo.set_servo_angle(SERVO_ID, -90.0, power=400) # 设置舵机角度
# (指定功率 单位mW)
uservo.wait() # 等待舵机静止

#####

```



```
#####

print("[多圈模式]设置舵机角度为900.0°，周期1000ms")
uservo.set_servo_angle(SERVO_ID, 900.0, interval=1000,
is_mturn=True) # 设置舵机角度(指定周期 单位ms)
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))

print("[多圈模式]设置舵机角度为-900.0°，设置转速为200 °/s")
uservo.set_servo_angle(SERVO_ID, -900.0, velocity=200.0,
t_acc=100, t_dec=100, is_mturn=True) # 设置舵机角度(指定转速 单位°/s) dps: degree per second
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))

print("[多圈模式]设置舵机角度为-850.0°，添加功率限制")
uservo.set_servo_angle(SERVO_ID, -850.0, power=400,
is_mturn=True) # 设置舵机角度(指定功率 单位mW)
uservo.wait() # 等待舵机静止
print("-> {}".format(uservo.query_servo_angle(SERVO_ID)))
```

轮转模式

API-wheel_stop

轮转模式停止转动。

函数原型

```
def wheel_stop(self, servo_id):
```

输入参数

- servo_id: 舵机ID

输出参数

- 无

API-set_wheel_norm

设置轮转普通模式，转速单位: %s

函数原型

```
def set_wheel_norm(self, servo_id, is_cw=True, mean_dps=None)
```

输入参数

- `servo_id`: 舵机ID
- `is_cw`: 是否是顺时针
- `True`: 顺时针
- `False`: 逆时针
- `mean_dps`: 平均转速

输出参数

- 无

API-set_wheel_turn

轮转模式，让舵机旋转特定的圈数。

函数原型

```
def set_wheel_turn(self, servo_id, turn=1, is_cw=True, mean_dps=None, is_wait=True):
```

输入参数

- `servo_id`: 舵机ID
- `turn`: 目标要旋转的圈数
- `is_cw`: 旋转方向，是否为顺时针
- `True`: 顺时针
- `False`: 逆时针

- `mean_dps`: 平均转速
- `is_wait`: 是否是阻塞式等待

输出参数

- 无

API-set_wheel_time

轮转模式，旋转特定的时间。

函数原型

```
def set_wheel_time(self, servo_id, interval=1000, is_cw=True,
                    mean_dps=None, is_wait=True):
```

输入参数

- `servo_id`: 舵机ID
- `interval`: 目标要旋转的时间, 单位ms
- `is_cw`: 旋转方向，是否为顺时针
- `True`: 顺时针
- `False`: 逆时针
- `mean_dps`: 平均转速, 单位dps
- `is_wait`: 是否是阻塞式等待

输出参数

- 无

例程源码

`src/wheel.py`

```
...
FashionStar Uart舵机
```

> Python SDK 舵机轮转模式测试 <

```
-----  
-  
-----  
...  

```

添加uservo.py的系统路径

```
import sys  
sys.path.append(".././src")  
# 导入依赖  
import time  
import serial  
from uservo import UartServoManager
```

参数配置

角度定义

SERVO_PORT_NAME = 'COM7' # 舵机串口号

SERVO_BAUDRATE = 115200 # 舵机的波特率

SERVO_ID = 0 # 舵机的ID号

初始化串口

```
uart = serial.Serial(port=SERVO_PORT_NAME,  
baudrate=SERVO_BAUDRATE,\n                        parity=serial.PARITY_NONE, stopbits=1,\n                        bytesize=8, timeout=0)
```

初始化舵机管理器

uservo = UartServoManager(uart)

print("测试常规模式")

设置舵机为轮转普通模式

旋转方向(is_cw) : 顺时针

角速度(mean_dps) : 单位°/s

uservo.set_wheel_norm(SERVO_ID, is_cw=True, mean_dps=200.0)

延时5s然后关闭

time.sleep(5.0)

停止

uservo.wheel_stop(SERVO_ID)

time.sleep(1)

定圈模式

print("测试定圈模式")

uservo.set_wheel_turn(SERVO_ID, turn=5, is_cw=False,
mean_dps=200.0)

定时模式

print("测试定时模式")

```
uservo.set_wheel_time(SERVO_ID, interval=5000, is_cw=True,  
mean_dps=200.0)
```

用户配置表修改

API-reset_user_data

重置用户数据表, 恢复默认值。

函数原型

```
def reset_user_data(self, servo_id):
```

输入参数

- `servo_id`: 舵机ID

输出参数

- 无

API-read_data

读取数据。

函数原型

```
def read_data(self, servo_id, address):
```

输入参数

- `servo_id`: 舵机ID
- `address`: 内存表

输出参数

- `content`: 数值的二进制数据流

API-write_data

写入数据。

函数原型

```
def write_data(self, servo_id, address, content):
```

输入参数

- `servo_id`: 舵机ID
- `address`: 内存表
- `content`: 数值的二进制数据流

输出参数

- 无

例程源码-重置用户数据表

example/reset_user_data.py

```
'''
FashionStar Uart舵机
> 内存表数据重置 <

注意事项：重置内存表这个指令比较特殊，舵机ID也会被重置为0
因此测试该指令的时候，最好只接一颗舵机。
-----
-
-----
'''

# 添加uservo.py的系统路径
import sys
sys.path.append("../src")
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager
```

```

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 数据表定义
ADDRESS_SOFTSTART = 49 # 上电缓启动地址位
SOFTSTART_OPEN = 1 # 上电缓启动-开启
SOFTSTART_CLOSE = 0 # 上电缓启动-关闭

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                      baudrate=SERVO_BAUDRATE, \
                      parity=serial.PARITY_NONE, stopbits=1, \
                      bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart, is_debug=True)
# 重置用户数据
uservo.reset_user_data(SERVO_ID)

# 舵机扫描
print("开始进行舵机扫描")
uservo.scan_servo()
servo_list = list(uservo.servos.keys())
print("舵机扫描结束, 舵机列表: {}".format(servo_list))

if SERVO_ID not in servo_list:
    print("指定的SERVO_ID无效, 请修改舵机ID列表")
    exit(-1)

print("重置舵机内存表: 舵机ID = {}".format(SERVO_ID))
uservo.reset_user_data(SERVO_ID)

print("重新进行舵机扫描")
uservo.scan_servo()
servo_list = list(uservo.servos.keys())
print("舵机扫描结束, 舵机列表: {}".format(servo_list))

```

例程源码-读取内存表

example/read_data.py

```

'''
FashionStar Uart舵机
> 内存表数据读取 <
-----
-
-----
'''

# 添加uservo.py的系统路径
import sys
sys.path.append("../src")
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号
# 数据表定义
ADDRESS_VOLTAGE = 1 # 总线电压值的地址

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
baudrate=SERVO_BAUDRATE, \
                    parity=serial.PARITY_NONE, stopbits=1, \
                    bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

# 内存表读取
# 注：因为每个数据位数据格式各不相同
# 因此读取得到的是字节流
voltage_bytes = uservo.read_data(SERVO_ID, ADDRESS_VOLTAGE)
# 数据解析
# 电压的数据格式为uint16_t, 单位：mV
# 关于struct的用法，请参阅官方手册：
https://docs.python.org/3/library/struct.html
voltage = struct.unpack('<H', voltage_bytes)

print("总线电压 {} mV".format(voltage))

```

例程源码-写入内存表

example/write_data.py

```
'''
FashionStar Uart舵机
> 内存表数据写入 <
-----
-
-----
'''

# 添加uservo.py的系统路径
import sys
sys.path.append(".././src")
# 导入依赖
import time
import struct
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 数据表定义
ADDRESS_SOFTSTART = 49 # 上电缓启动地址位
SOFTSTART_OPEN = 1 # 上电缓启动-开启
SOFTSTART_CLOSE = 0 # 上电缓启动-关闭

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
                      baudrate=SERVO_BAUDRATE, \
                      parity=serial.PARITY_NONE, stopbits=1, \
                      bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

# 内存表写入
# 注：在写入之前，需要查阅手册确保该数据位可写
# 缓启动数据类型 uint8_t, 首先构造数据位
softstart_bytes = struct.pack('<B', SOFTSTART_OPEN)
# 将数据写入内存表
ret = uservo.write_data(SERVO_ID, ADDRESS_SOFTSTART,
                        softstart_bytes)
# 打印日志
print("缓启动数据写入是否成功：{}".format(ret))
```

系统状态查询

API-query_voltage

查询当前的电压

函数原型

```
def query_voltage(self, servo_id)
```

输入参数

- `servo_id`: 舵机ID

输出参数

- `voltage`: 电压, 单位V

API-query_current

查询当前的电流

函数原型

```
def query_current(self, servo_id):
```

输入参数

- `servo_id`: 舵机ID

输出参数

- `power`: 舵机电流, 单位A

API-query_power

查询当前的功率

函数原型

```
def query_power(self, servo_id)
```

输入参数

- `servo_id`: 舵机ID

输出参数

- `power`: 舵机功率, 单位W

API-query_temperature

查询舵机当前的温度

函数原型

```
def query_temperature(self, servo_id)
```

输入参数

- `servo_id`: 舵机ID

输出参数

- `temperature`: 温度, ADC值

例程源码

example/servo_status.py

```
'''
FashionStar Uart舵机
> 读取舵机的状态信息 <
-----
-
-----
'''
```

```

# 添加uservo.py的系统路径
import sys
sys.path.append("../src")
# 导入依赖
import time
import serial
from uservo import UartServoManager

# 参数配置
# 角度定义
SERVO_PORT_NAME = 'COM7' # 舵机串口号
SERVO_BAUDRATE = 115200 # 舵机的波特率
SERVO_ID = 0 # 舵机的ID号

# 初始化串口
uart = serial.Serial(port=SERVO_PORT_NAME,
baudrate=SERVO_BAUDRATE, \
                    parity=serial.PARITY_NONE, stopbits=1, \
                    bytesize=8, timeout=0)

# 初始化舵机管理器
uservo = UartServoManager(uart)

def log_servo_status():
    '''打印舵机状态'''
    # 读取温度
    voltage = uservo.query_voltage(SERVO_ID)
    # 读取电流
    current = uservo.query_current(SERVO_ID)
    # 读取功率
    power = uservo.query_power(SERVO_ID)
    # 读取温度
    temp = uservo.query_temperature(SERVO_ID)

    print("Voltage: {:.41f}V; Current: {:.41f}A; Power:
{:4.1f}W; T: {:.2.0f}".format(\
        voltage, current, power, temp), end='\r')

while True:
    uservo.set_servo_angle(SERVO_ID, 90)
    while not uservo.is_stop():
        log_servo_status()
        time.sleep(0.1)

    time.sleep(1)

    uservo.set_servo_angle(SERVO_ID, -90)
    while not uservo.is_stop():
        log_servo_status()
        time.sleep(0.1)

```

```
time.sleep(1)
```

注：ADC转换为摄氏度值公式

$R1 = 10000;$ (NTC 分压电阻值, $R1$ 靠近电源, NTC 靠近 GND)

$Rt25 = 10000;$ (NTC 在 25° 室温时的电阻值)

$B = 3435;$ (NTC 的材料常数)

$K = 273.15;$ (绝对零度)

$T = 1/(\ln(R1 \cdot ADC \div Rt25 \div (4096 - ADC)) \div B + 1 \div (K + 25)) - K$

温度(°C)	ADC	温度(°C)	ADC	温度(°C)	ADC
50	1191	60	941	70	741
51	1164	61	918	71	723
52	1137	62	897	72	706
53	1110	63	876	73	689
54	1085	64	855	74	673
55	1059	65	835	75	657
56	1034	66	815	76	642
57	1010	67	796	77	627
58	986	68	777	78	612
59	963	69	759	79	598

- 以上为50-79°C 温度/ADC参照表

